

Relazione di Progetto — Drafts

Piattaforma di Commissioni Artistiche

Esame di Applicazioni Web — A.A. 2025/2026

1. Descrizione generale

Drafts è un'applicazione web che mette in contatto **clienti** e **artisti** per la creazione di opere su commissione. Un cliente sceglie un artista, richiede un servizio, paga, riceve il lavoro consegnato e lascia una recensione. L'artista gestisce il proprio portfolio, i propri servizi e le commissioni in arrivo. Un amministratore modera utenti, categorie e recensioni.

L'applicazione gestisce l'intero ciclo di vita di una commissione, dal pagamento online (Stripe) alla consegna del file fino alla recensione finale.

2. Architettura e scelte progettuali

2.1 Approccio SPA (client + server)

È stato scelto l'approccio **Single Page Application**: il server espone una **API REST** in formato JSON, mentre il **client** costruisce dinamicamente l'HTML a partire dai dati ricevuti. Di conseguenza **non viene utilizzato EJS** (templating lato server), che è alternativo a questo approccio. Il routing lato client è gestito con **page.js**. Questa scelta separa nettamente back-end (dati e logica) e front-end (presentazione), favorendo modularità e riuso del codice.

2.2 Target dispositivo

L'applicazione è progettata **desktop-first** con supporto **responsive**: su schermi piccoli la navbar a "pillole" lascia il posto a una barra superiore con drawer laterale (menu a scomparsa), e le griglie passano a layout a colonna singola.

2.3 Stack tecnologico

Back-end

- Node.js + **Express 5**
- **SQLite3** (database relazionale su file `drafts.db`)
- **Passport** (strategia `local`) + **express-session** per l'autenticazione
- **bcrypt** per l'hashing delle password
- **multer** per l'upload dei file immagine
- **stripe** per i pagamenti online

- dotenv, morgan

Front-end

- JavaScript a oggetti (**classi ES6**), organizzato in moduli
- **page.js** per il routing lato client
- **Bootstrap 5** + Bootstrap Icons per l'impalcatura UI
- **Swiper** per i caroselli
- Google Fonts (Sora per i titoli, Inter per il testo)
- CSS3 personalizzato con **variabili CSS** (:root) per la palette

3. Struttura del database

Il database relazionale SQLite è composto dalle seguenti tabelle (schema in database.sql):

Tabella	Descrizione
users	utenti con ruolo (client, artist, admin), email, password hashata
artists	dati estesi dell'artista (nome d'arte, città, bio, immagine profilo); 1■a■1 con users
categories	categorie artistiche (es. Illustrazione, Concept Art)
portfolios	opere dell'artista (nome, immagine, descrizione, categoria)
services	servizi offerti (nome, prezzo, giorni di consegna, categoria)
commissions	commissioni con stato, prezzo, dati pagamento Stripe e immagine di consegna
reviews	recensioni (voto + testo) legate a una commissione consegnata
favorites	relazione molti-a-molti tra client e artisti preferiti
notifications	notifiche per utente (messaggio, letta/non letta)

Relazioni principali

- Un user con ruolo artista ha una riga in artists (FK user_id).
- portfolios e services appartengono a un artista (FK artist_id) e possono riferire una category (FK category_id, ON DELETE SET NULL).
- commissions collega un cliente, un artista e un servizio.
- reviews è legata 1■a■1 a una commissione (una recensione per commissione).
- favorites è una tabella ponte (client_id, artist_id).
- Le foreign key usano ON DELETE CASCADE o SET NULL per mantenere l'integrità.

Macchina a stati della commissione

```
pending -> accepted / rejected
accepted -> in_progress    (solo dopo pagamento Stripe)
in_progress -> delivered   (consegna immagine)
delivered -> reviewed      (recensione del cliente)
```

La transizione accepted -> in_progress avviene **esclusivamente** tramite pagamento, non manualmente dall'artista (doppio controllo lato middleware e lato UI).

4. Back-end: API e logica

L'API è organizzata per risorsa in `server/routes/api/`. Ogni risorsa ha un **DAO** (`server/dao/`) che incapsula le query SQL e mappa le righe in oggetti.

I **metodi HTTP** sono usati in modo semantico:

- GET per leggere, POST per creare, PUT per aggiornare, PATCH per cambi di stato,

DELETE per eliminare.

Principali endpoint: `/api/sessions` (login/logout), `/api/users`, `/api/users/artists`, `/api/categories`, `/api/portfolios`, `/api/services`, `/api/commissions`, `/api/reviews`, `/api/favorites`, `/api/uploads`, `/api/payments`, `/api/notifications`.

Autorizzazione

L'accesso è regolato da **middleware** dedicati: `requireAuth`, `hasRole`, e middleware specifici (es. `isInvolved`, `canDelete`, `canChangeStatus` per le commissioni). I dati sensibili sono filtrati per ruolo: un cliente vede solo le proprie commissioni, un artista solo quelle a lui rivolte, l'admin tutte.

5. Front-end: componenti e layout

Il front-end è organizzato in **classi ES6**, una per vista/componente, in `public/components/`. Ogni vista segue lo stesso pattern: `render()` (fetch + spinner + inserimento template + binding eventi), metodi `template()`/`*Template()` e `attachEvents()`.

Componenti persistenti (renderizzati una volta, fuori dalla `<main>`):

- **NavBar** — navigazione, ricerca artisti con dropdown live, menu utente, campanella notifiche
- **Footer** — presente su tutte le pagine

Viste principali:

- `HomeView` — hero + carosello "Top artisti"
- `ArtistListView` — lista artisti con sidebar filtri (città, rating, categoria) + preferiti

- `ArtistDetailView` — profilo artista a schede (Portfolio, Servizi, Recensioni)
- `ServicesView` — catalogo globale dei servizi filtrabile per categoria/nome
- `CategoryView` — artisti e opere di una categoria
- `MyServicesView`, `MyPortfolioView` — gestione CRUD per l'artista
- `MyCommissionsView` — dashboard commissioni (pagamento, consegna, recensione)
- `MyReviewsView`, `FavoritesView`, `ProfileView`, `AdminView`
- Modali: `LoginView`, `SignupView`, `CommissionModal`, `DeliveryModal`

Scelte di layout/UI:

- Navbar "a pillole" con effetto vetro (glassmorphism), ispirata a interfacce moderne
- Palette teal/blu definita con variabili CSS, facilmente modificabile in un punto unico
- Caricamento file con **drag & drop** e barra di avanzamento (`DeliveryModal`)
- Overlay descrizione su hover, badge di stato colorati, avatar con iniziale o immagine

6. Funzionalità principali

- **Autenticazione** con sessione e ruoli (client/artist/admin)
- **Ricerca e filtri** di artisti e servizi (per categoria, città, rating, nome)
- **Preferiti** (toggle cuore) per i clienti
- **CRUD** di portfolio e servizi per gli artisti, con **upload immagini reali**
- **Flusso commissione completo**: richiesta -> accettazione -> **pagamento Stripe** ->

consegna del file -> recensione

- **Notifiche** in app con polling (near realtime) su tutti gli eventi del flusso
- **Pannello admin** per moderazione di utenti, categorie e recensioni

7. Integrazioni esterne (concordate con i docenti)

- **Stripe Checkout** — pagamento online della commissione in modalità test. Il client viene reindirizzato alla pagina di pagamento ospitata da Stripe; al ritorno il server verifica l'esito e sblocca la commissione.
- **multer** — upload di immagini (portfolio, immagine profilo, consegna lavoro) salvate in `public/uploads` e servite staticamente.

8. Sicurezza

- **Password**: salvate con hash **bcrypt**, mai in chiaro.

- **SQL injection:** prevenuta con **query parametrizzate** (? + array di parametri) in tutti i DAO.
 - **XSS:** i dati inseriti dagli utenti vengono filtrati con una funzione `escapeHtml` prima di essere inseriti nel DOM; gli URL delle immagini sono validati lato server.
 - **Sessioni:** cookie `httpOnly`, autorizzazione per ruolo sulle rotte protette.
 - **Filtro per ruolo:** le commissioni sono visibili solo agli utenti coinvolti.
-

9. Utenti e dispositivi di test

- Compatibile con le ultime versioni di **Chrome** e **Firefox**.
- L'applicazione si avvia con `npm start` (server su `http://localhost:3001`).
- Le credenziali Stripe di test sono configurate nel file `.env`.